

APPSC Intelligent Newspaper-to-Exam Preparation System

How one uploaded newspaper becomes exam-shaped notes and questions for the three APPSC papers that are answered by ticking a box — the complete process, architecture, data model, constraints and current state.

Codebase	appsc-current-affairs-app
Compiled	23 August 2026, from the source and the live database
Serves	Group-II Screening · Group-II Mains · Group-I Prelims
Verified by	146/146 tests · 3 static gates · preflight 8 pass / 2 warn / 0 fail
Content state	4 editions · 411 articles · 119 published items · 874 questions
Supersedes	Version 1.0, written before the Group-I Mains layer was removed

Contents

PART I — WHAT THIS SYSTEM IS

- 1 The problem it solves — why a newspaper is the wrong shape for an exam
- 2 The exam being served — three papers, all of them ticked
- 3 The core principle — one source, one intelligence process, many outputs
- 4 Dual routing — one reading, two published syllabi

PART II — THE PROCESS, END TO END

- 5 The journey of one PDF — eight stages, and which ones cost money
- 6 Stages 1–2 — extraction and segmentation
- 7 Stage 3 — the relevance score
- 8 Stage 4 — adaptive selection by syllabus leverage
- 9 Stage 5 — drafting the master knowledge object
- 10 Stage 6 — questions written to the syllabus unit
- 11 Stages 7–8 — review and publication

PART III — ARCHITECTURE

- 12 Stack, processes and the request path
- 13 Repository map
- 14 The data model — thirty-five tables, by layer
- 15 The API surface

PART IV — THE INTELLIGENCE LAYERS

- 16 The ten layers, and what is actually built
- 17 The syllabus map and its aliases
- 18 The PYQ engine — what APPSC has really asked
- 19 The topic knowledge graph

PART V — THE PRODUCT

- 20 Student screens
- 21 Admin screens
- 22 The gates — everything that must pass before a student sees a word

PART VI — OPERATIONS

- 23 Setup, running, and the daily commands
- 24 Configuration
- 25 Preflight, backup, deploy, restore
- 26 Tests and static gates

PART VII — CONSTRAINTS, LIMITS AND STATE

27	Standing decisions
28	Invariants the code enforces
29	Known limitations
30	Open backlog
31	Current content state — live figures
32	Decision log — failures that shaped the design
33	What the Group-I Mains removal took out
34	Glossary

What this system is

Before any code: the problem, the exam, the one principle everything else follows from, and the single design idea that makes the app different from a notes app with a newspaper attached.

1. The problem it solves

A newspaper is organised by the day. An exam is organised by the syllabus. Every hour a candidate spends converting the first into the second is an hour not spent learning, and it is the hour almost nobody sustains for nine months.

The daily current-affairs habit fails in a specific, predictable way. A candidate reads the paper, underlines things, and produces a pile of dated notes. Eight months later the pile is unusable: it is filed by date, and the question paper does not ask about 21 August. It asks about the Finance Commission — and the four separate days on which the Finance Commission appeared are four separate pages in four separate notebooks, with no thread between them.

Worse, the same candidate is usually preparing for **two exams** with two different published syllabi. The same event belongs to different units in each, and reading the paper twice — once per syllabus — is the advice everyone gives and nobody follows.

This system takes the conversion work off the candidate. The admin uploads the day's paper as a PDF; the system extracts every article, scores each one against the actual APPSC syllabus, drafts the survivors into exam-shaped notes, writes questions in the formats APPSC actually uses, and files everything against syllabus units and persistent topics — so that the fourth Finance Commission story attaches to the first three instead of starting a new page.

THE CLAIM, STATED PLAINLY

The competitive advantage is **not** OCR and **not** AI note generation. Both are commodities. The advantage is the chain:

Current Event → APPSC Relevance → Syllabus Topic → Blueprint Keyword → PYQ Pattern → Cross-Paper Connection → AP-Specific Link → Correct Exam Output.

2. The exam being served

Andhra Pradesh Public Service Commission runs four papers a candidate at this level might sit. **This app serves the three that are answered by ticking a box.**

Paper	Answer shape	What it rewards	Units mapped here
-------	--------------	-----------------	-------------------

Group-II Screening	Objective	Fact recall, heavy on current affairs	25 units, shared
Group-II Mains	Objective	The same, deeper, across three papers	with Screening
Group-I Prelims	Objective	Precise recall across a wide syllabus, 120 questions	27 units
Group-I Mains	Descriptive	An argument, structured, with an AP example in it — out of scope since 23 Aug 2026	REMOVED

THE DECISION THAT SHAPES THIS WHOLE DOCUMENT

Serving the written paper as well meant carrying a **second output shape through every layer**: an eight-section note per item, four capture banks, essay questions, a seven-dimension tagging scheme, 54 descriptive syllabus units and a 252-question descriptive past-paper corpus — none of which a ticked paper uses.

All of it was removed on 23 August 2026. What remains is one intelligence process feeding two objective syllabi. **Section 33 records exactly what went**, so the decision can be re-read rather than re-derived.

This removes Group-I *Mains*. It does not remove Group-I: Prelims is served, and its 27 units are more than half the syllabus map.

Why question generation is the main output, not a bolt-on

Every paper is objective, so the memorise-this block and the questions written from it are not a lane — they are the product. The number of questions an item gets follows **how many distinct syllabus units the article feeds**, from a base of 4 up to a cap of 10, rather than a flat four per item.

Why Andhra Pradesh is a separate axis, not a subject

AP content runs through every paper rather than sitting in one unit. So the system treats AP as a dimension that cuts across everything: articles carry an `ap` flag, the relevance score gives AP its own 20 of 100, and the four content buckets include `ap` as a peer of `national`. Of the 119 items currently published, **45 are AP** — 38%.

The strongest single signal that a story is an AP story turns out not to be its text but its **dateline**. A report filed from AMALAPURAM is an AP story even when the body never names the State, which is why `np_articles.dateline` is a first-class column and why a bug that once wiped it was treated as a data-loss incident rather than a cosmetic one.

3. The core principle

ONE SOURCE → ONE INTELLIGENCE PROCESS → MANY EXAM OUTPUTS

NEWSPAPER

- ARTICLE EXTRACTION
- FACT + EVENT UNDERSTANDING
- MASTER APPSC KNOWLEDGE OBJECT
- SYLLABUS / PYQ / KEYWORD MAPPING
- MULTIPLE EXAM-SPECIFIC OUTPUTS

The newspaper is processed **once**. The same processed knowledge then serves both syllabi. There is deliberately no "Newspaper → Group I" pipeline sitting beside a "Newspaper → Group II" pipeline, because that duplicates the expensive part and guarantees the two will disagree.

This principle has a concrete architectural consequence that recurs throughout the codebase: **shared libraries, not parallel implementations**. The PDF path used by the admin upload screen and the path used by the command line are the same function (`lib/ingest.js`). The drafting insert used by the newspaper lane and by the older web lane is the same function (`insertDrafted` in `lib/draft.js`). Where a second copy was tempting, the reasoning written into the source is always the same: two copies drift, and the drift is silent.

4. Dual routing — one reading, two published syllabi

The atomic object is **one item, routed to both syllabi at once**. A student reads a day's items through a *lens* they switch in the header. Read once, file twice.

WHAT THE LENS MEANS NOW

It used to switch between a *written* paper and a *ticked* one — two answer shapes. Both exams are ticked now, so it switches between **two syllabi**. The facts are the same; what differs is where on each published syllabus the item sits.

Worked example, item 100, under the "both" lens:

GROUP II → G2-S2 Geography — physical, economic and human, India and AP

GROUP I PRELIMS → G1P-D1 General geography — earth, atmosphere, oceans, climate change, hazards

	Group-II lane	Group-I Prelims lane
Atomic output	An MCQ in one of 8 official formats — shared	
Recall block	<code>prelims_facts</code> — shared	
Keyword angles	Blueprint angles — <code>Appointed</code> , <code>G1 tag</code> , <code>Index</code> — shared	
Syllabus units	G2-* — 25 units across 5 papers	G1P-* — 27 units across 6 subject papers
Grouping	4 buckets: International / National / AP / dynamic	

What every item carries

SHARED headline · event date · bucket · importance tier ·
 notes_markdown · static_linkage · static_notes · sources

RECALL prelims_facts the memorise-this block, and the whole output
 keywords[] blueprint question angles
 units[] every syllabus unit it feeds, in both exams
 mcqs[] 8 formats, each tagged format + keyword + unit

THREE RULES THE CODE ACTUALLY ENFORCES

1. No facts, no item. An item with no memorise-this block is an item no question can be written from, and every paper here is answered by ticking a box. The drafter turns it away before spending anything more on it, and the review queue flags it in red. (*This replaced the old "no angle, no Group-I lane" rule, which guarded the written paper.*)

2. Discard is a first-class outcome. Most news should be discarded. Discards are kept as rows with their reason, and the admin dashboard shows the count as a *positive* signal. Currently 36 of 155 drafted items and 64 of 411 articles are discarded rows, not deletions.

3. Facts go stale, so say when they were true. Every question carries `fact_as_of`, every explanation is required, and a seeded `ref_corrections` register is both injected into the drafting prompt *and* checked against every draft. A wrong current-affairs fact cannot be caught against a textbook, which is why this gets three layers rather than one.

The process, end to end

Follow one uploaded PDF all the way to a student's screen. Eight stages, five of which involve no model at all, and two of which cost money. This is the part to understand first — everything in Parts III and IV is machinery in service of it.

5. The journey of one PDF

ADMIN UPLOADS THE HINDU (VIJAYAWADA), 21 AUG 2026, 28 PAGES, ~120 MB

- | | | |
|-------------|-----------------|--|
| 1 EXTRACT | <i>no model</i> | → text with bounding boxes; OCR only where there is no text layer |
| 2 SEGMENT | <i>no model</i> | → blocks become articles; genre read from the paper's own running head |
| 3 SCORE | <i>no model</i> | → 0–100 on five weighted factors, with the breakdown stored |
| 4 SELECT | <i>no model</i> | → rank by syllabus leverage; decide what is worth drafting |
| 5 DRAFT | model | → the master knowledge object: notes, static notes, the facts block |
| 6 QUESTIONS | model | → MCQs in the formats APPSC has actually used for that angle |
| 7 REVIEW | <i>a person</i> | → the review queue; sources, tags and correction hits shown inline |
| 8 PUBLISH | <i>a person</i> | → the day goes live; students see it |

Nothing between stage 1 and stage 8 reaches a student. Publishing is always manual.

Where the money goes, and where it does not

Five of the eight stages are deterministic code. That is a deliberate cost and trust decision, not an accident of implementation:

- **Free stages are repeatable.** Re-scoring an edition after improving the vocabulary costs nothing, so the vocabulary can be improved often.
- **Free stages are explainable.** "Why did this article drop out of today's list?" has an answer — the stored breakdown — rather than a shrug.
- **Free stages are stable.** A model asked to score relevance gives a different answer to the same article on Tuesday.
- **Paid stages do what only a model can** — write the note prose and the standing material underneath it.

The practical consequence: you can inspect the entire selection for an edition, for free, in under a second, before spending anything:

```
node server/scripts/draft-articles.js 4 --plan
```


`--plan` prints what would be drafted and stops. It is distinct from `--dry-run`, which drafts everything and declines to save it — same money, same twenty minutes.

6. Stages 1–2 — extraction and segmentation

Extraction

A Python helper (`content-pipeline/np-daily/layout.py`) reads the PDF and returns text *with bounding boxes*. Position is the whole point: a newspaper page is a layout problem, not a text problem, and a plain text dump interleaves columns into nonsense. Pages that already carry a text layer are read directly; only pages without one are rasterised and OCR'd, which keeps a 28-page edition affordable in time. The counts of OCR'd and skipped pages are stored on the edition row, because a run that silently OCR'd everything looks identical to one that did not.

The upload itself is **idempotent**: an edition is identified by publication + date + a SHA-256 of the file bytes, so uploading the same PDF twice returns the existing edition rather than creating a second one. Processing is the expensive part; a double-clicked upload button should not cost two OCR passes.

Segmentation — one rule, and no column grid

THE SEGMENTATION RULE

A body block belongs to **the nearest headline above it whose horizontal span covers the block's centre**.

That works because of how pages are designed rather than by luck. A headline placed in a column is exactly the mark that the story above it ended there. A six-column headline owns six columns of text; a one-column headline owns one. The rule was checked by hand against all twelve ambiguous blocks on a real page that sets its upper half and its lower strip on different column pitches — the case where a naive column grid fails.

Genre comes from the paper, not from a classifier

The Hindu names its own pages in the running head, so "Editorial", "Opinion" and the `PARLEY` and `NOTEBOOK` kickers are read directly off the page. This matters because **the same sentence is a fact in a report and a claim in an op-ed**. An item drafted from signed opinion gets a different prompt, a forced `needs_verify` flag, and the author's name carried onto the page.

Same-event merging

Two reports of one event become one enriched item. A duplicate is not deleted — it points at the article it duplicates via `merged_into`, so a wrong merge can be inspected and undone. Jump continuations ("continued on page 12") are folded into the story they continue. Currently 14 of 411 articles are marked `duplicate`.

7. Stage 3 — the relevance score

Every article gets a 0–100 score with the breakdown stored alongside it. Five weighted factors:

Factor	Measures	Weight	How it is earned
A	Syllabus relevance	30	Connects to a known master topic, or to recognisable subject territory, or to a syllabus unit. A Tier-1 topic tops the factor out.
B	PYQ keyword match	20	Matches a blueprint angle the commission has <i>actually asked before</i> — the count comes from the PYQ corpus, not from a hunch.
C	Andhra Pradesh	20	Full marks for an AP story (the dateline counts); half marks where a topic is AP-specific but the article reads as national.
D	Current importance	15	Does it name a findable official act? The difference between a decision that can be cited and a meeting that was held.
E	Cross-paper reuse	15	How many distinct <i>objective</i> papers the matched topics serve — study once, tick it off in three places. Re-based 23 Aug 2026; see below.

Bands: 80–100 CRITICAL · 60–79 HIGH · 40–59 MEDIUM · below 40 LOW.

Why every score carries its breakdown

Because a number nobody can decompose is a number nobody trusts. The first question anyone asks about "62 / HIGH" is which part came from where, and if that cannot be answered the score gets ignored and the list goes back to being read top to bottom. So `score()` returns the five components *and the reason each one fired*, and the caller stores them in `np_articles.breakdown`. It also makes the weights testable: 30 for syllabus against 20 for AP is an opinion about this exam, and an opinion that can be re-run over past articles can be corrected.

FACTOR E WAS LOAD-BEARING, AND NEARLY WENT SILENTLY

It read `topic_evidence` — 149 rows holding one person's reading of the Group-I **Mains** papers, and *nothing else*. Removing the Mains layer would have taken **15 of the 100 points off every article**, shifting the whole ranking, with no line anywhere saying why.

It is now measured from this app's own evidence: `topic_units` joined to `ref_units`, counting the distinct *objective* papers a topic's units belong to. It answers the same question the blueprint was being asked — "does studying this pay in more than one place?" — from data that can be rebuilt rather than from a document.

It now reaches further than the blueprint did: 26 articles earned Factor E before, 75 do now.

The thresholds were rescaled with it. Five Mains papers became **ten** objective ones, so leaving `4+ → 15` would have quietly redefined full marks from 80% breadth to 40%. Now `6+ → 15`, `4-5 → 12`, `3 → 8`, `2 → 4`. Measured effect is small and worth stating plainly: mean score `31.7 → 31.3` across 411 articles, three articles leaving the 70+ band. It was done anyway, because a threshold that no longer means what its comment says is how a score stops being trusted.

The veto comes first

WHY SOME ARTICLES ARE EXCLUDED BEFORE SCORING, NOT PENALISED DURING IT

A robbery in Nandyal mentions an AP district, names a police officer, and matches the *Arrested* angle: it would score in the forties on the factors alone. So categories that are never examinable are excluded *before* any scoring happens, rather than being outvoted by it.

The veto list covers letters to the editor, the archive columns ("A HUNDRED YEARS AGO", "FROM THE ARCHIVES" — reprints of copy a century old that read as perfectly plausible business reporting), crime reports, and sport. Sport is tested separately because the rule is narrower: a match report is not examinable, but doping, sports governance, a major tournament or a sports policy question is. Omitting that check once let a CBSE archery tournament reach 62/HIGH.

Guards learned from real misfires

The misfire	The guard now in place
"Collectors empowered to grant citizenship under CAA" filed as <i>international</i> , because a CAA story naturally mentions Pakistan and Bangladesh	International requires India-as-party framing, not merely a country name; and it must <i>out-signal</i> the domestic reading rather than merely appear in it. A tie goes to national — this is a State exam.
"Imran Khan shifted back to jail" matched the Tier-1 topic "Supreme Court and judicial review" on the words <i>Supreme Court</i>	A foreign-affairs guard: a topic match inside another country's internal affairs is naming the wrong country's institution, so topic-derived tiers are withheld.
"Woman dies in lift accident" rose from 20 to 30 on the word <i>hospital</i> ; a Ukraine war report on <i>missile</i>	A unit must be named in the headline , or reached by two distinct terms in the body, or reached by one multi-word phrase specific enough to stand alone.
Requiring two terms then read "YSRCP disrupts proceedings in Council" as off-syllabus, though the body says <i>Legislative Council</i> once and says it exactly right	The single-phrase escape above. Every alias that misfired was a single common word; every phrase that worked — "Legislative Council", "minimum support price" — names its unit on sight.

Measured distribution

Across the 411 articles currently scored:

Band	Articles	Share
70 and above	30	7%
55–69	40	10%
45–54	43	10%
32–44	88	21%
below 32	210	51%

Half the newspaper scores under 32. That is the system working: most of a newspaper is not examinable, and a pipeline that finds otherwise has stopped filtering.

8. Stage 4 — adaptive selection by syllabus leverage

This stage decides which scored articles are worth paying to draft.

What was wrong with a flat threshold

The original rule was `score >= 45`. Because the score is a *blend* of five factors, an article can clear the bar on AP place-names and importance alone while feeding no syllabus unit at all — and an article that plainly belongs to a unit can miss the bar because nothing else about it scored. Measured across 248 scored articles at the time:

Under the flat rule	Articles	Outcome
Scored 45+ and feed no syllabus unit	10	Drafted anyway — paid for, and unfileable
Scored under 45 and do feed a unit	54	Never drafted — the expensive miss

Wrong in both directions, and the second number is the costly one. "Centre notifies key scheme to manufacture mobile phones" scored 43 and feeds four units. "Ethanol blending policy is behind rise in sugar prices" scored 39 and feeds six.

The lever

The number of **distinct syllabus units** an article feeds is the signal the composite buries. In the 35–44 band, 9 of the 20 articles that match at all match three or more units. An article examinable in three places is examinable, whatever its composite came to.

```
leverage = 40 (1 unit) | 62 (2) | 76 (3) | 85 (4+)
           + 15 if a unit is named in the HEADLINE
           capped at 100

rank      = 0.55 × leverage + 0.45 × composite score
```

Two details carry judgement. **Leverage flattens after four units**, because the difference between four and seven is not "nearly twice as examinable" — past a point, a long unit list is a story touching many things lightly, which is the shape of a default block rather than a strong match. And **headline placement beats a fourth unit**, because a unit named in the headline is what the story is *about*, not something it mentions.

The guards, each from something the data actually did

Guard	Value	Why
minRank	45	The rank an article must reach to be drafted at all.

minScore	32	A floor on the composite whatever the leverage says. "Karnataka govt. bans sale of non-dairy paneer" scored 22, touched three units, and ranked 52. Three loose matches on a weak story is not a story about three units. Set to 32 rather than 36 because 36 also removed the J&K census enumerators, SEBI on bond-platform advertising and the Great Nicobar clearance.
unmatchedMinScore	null	An article connecting to no syllabus unit is never drafted automatically. See below.
minItems	12	A thin paper should still yield a digest; the selector reaches further down the ranked list — but never into unmatched articles.
maxItems	35	A rich paper should not yield ninety items. An explicit <code>--max</code> overrides <code>minItems</code> rather than being silently raised by it.

The judgement call: no syllabus connection, no draft

WHY A SCORE THRESHOLD FOR UNMATCHED ARTICLES CANNOT WORK

The rule started as "draft an unmatched article if it scores well enough". It was abandoned, because the score is precisely what cannot separate these two:

- 70 Cultural diversity highlight of gala dinner in Vizag
- 55 ₹7,470 cr. cleared for infra works in ULBs under HAM

The first is filler. The second is real AP infrastructure. Both score highly for the same reasons — AP place names, money, officialdom — and neither connects to the syllabus. **Any cut-off that admits the second admits the first.**

So the rule stops guessing. The valuable ones are not lost: they are unmatched because the syllabus *vocabulary* has a gap, and every run prints them as a to-do list — **17 such articles exist right now**. Add the missing alias, re-seed, re-score, and they come back through the front door. Hand-picking with `--article ID,ID` remains available for genuine exceptions, where the gate is a person rather than a rule.

9. Stage 5 — drafting the master knowledge object

This is the first stage that calls a model, and the design question is not "what prompt" but **what the model is and is not asked**.

THE RULE THAT SHAPES THE WHOLE PROMPT

The model is **not** asked to re-derive the bucket, the keyword angles, the topics, or the syllabus units. Stages 2–4 already computed those deterministically and reproducibly. They are supplied to it as **findings**, not questions. Asking a model a question already settled gets you a second, disagreeing answer.

It is asked for exactly what a scoring function cannot produce: the note prose, the standing material underneath it, and the memorise-this block of facts that questions are later written from.

The prompt got a third smaller when the Mains layer went

THE 6,250-TOKEN BLOCK THAT IS SIMPLY GONE

The model used to be handed the whole descriptive unit vocabulary to choose from — the largest single block in every paid call. It is no longer asked for units at all, because the objective ones were already matched from the newspaper's own words.

That also removes a contradiction which produced the worst tagging in the corpus. Told nothing, the model returned fourteen units for one story, half of them objective codes copied back from the findings; told not to copy them, it returned three objective codes and no descriptive ones. A prompt saying one thing in its instructions and the opposite in its vocabulary — and the vocabulary wins. There is no vocabulary to contradict now.

What is assembled into the call

Block	Contents
The base prompt	The objective-only brief: the required fields, the compact fact-dense style, the discard rule.
Print addendum	Rules specific to newspaper sourcing — no invented URLs, citation built from the edition row, <code>needs_verify</code> forced on.
Syllabus addendum	The actual APPSC syllabus text for the units this article feeds, so the note is written <i>to the unit the commission examines</i> rather than to the topic in general.
Vocabulary	The 737 blueprint keyword angles and the corrections register. No unit listing.
Opinion addendum	Added only for signed opinion pieces: treat assertions as claims, attribute them, flag for verification.
Findings	Matched keywords with their PYQ counts, matched topics with tier and AP flag, past-paper evidence for the angle.
The article	Headline, standfirst, byline, dateline, body, page, genre.

The static blocks are assembled *before* the per-article content deliberately: providers cache by prefix, so a stable prefix across a 30-article run is a large cost reduction. The opinion block sits last, behind everything identical, so an op-ed does not fork the cached prefix for every call after it.

The fields the model returns

Field	What it is for
headline , event_date	What happened, and when — as distinct from which digest it lands in
bucket , subject_tag	International / national / AP / dynamic, and the home subject for a dynamic item
notes_markdown	The exam-oriented note, 120–250 words, with a table wherever there is a natural pairing — those pairings become list-matching questions
static_linkage	Which standing syllabus topic this news updates
static_notes	The standing material itself, 150–300 words, so the student need not go elsewhere
prelims_facts	The most important field. Bare facts, one per line, "Label — value". Scanned at revision time, not read. Every line is a question that can be written later
keywords	Blueprint question angles
importance , needs_verify , verify_note	Tier 1–3, and an acknowledged gap where a figure could not be confirmed

Provenance for print items

Print items carry **no invented URL**. The citation is written from the edition row — publication, edition, date, page — and anything the model returns in `sources` is dropped. `is_primary` is 0, because a newspaper is a secondary source. Every bridged item is `needs_verify = 1` : one print report is not the cross-check the research discipline requires.

Robustness the drafter had to learn

Failure	Fix now in place
29 articles lost in one run to <code>fetch</code> failed , scoring 61, 58, 57, 54...	A transient-error classifier walking the whole <code>cause</code> chain, exponential backoff with jitter, a request timeout, and a second-pass queue for failures at the end of the run.
A killed run lost 13 drafts that had already been paid for	Drafts are written incrementally, per record , not held in memory until the end.
A truncated JSON array discarded every question for a 61-scoring item	<code>salvageObjects()</code> recovers the complete objects from a truncated array rather than failing the whole parse.
A unit tag echoed back as "G2-P1-U7 — Union and State government..." matched nothing and vanished silently	<code>canonicalUnit()</code> normalises it back to the code, boundary-matched so U10 is never confused with U1. A unit code nothing can match is not a wrong tag — it is an invisible one.

10. Stage 6 — questions written to the syllabus unit

Question generation runs in the same worker, sharing the corrections guard, the validators and a corpus-wide duplicate hash.

The eight formats

Format	In the bank	Format	In the bank
direct_recall	257	negative_statement	82
list_matching	152	count_based	74
multi_statement	149	statement_based	73
assertion_reason	87	chronological	0

The format mix is driven by evidence, not a rotation

APPSC does not test every topic the same way. A Committee is asked as direct recall and list-matching; a constitutional Article as assertion-reason; a chronology as sequencing. The only way to know which is to count what the commission has actually done. `plannedFormatsFor()` asks: *given this blueprint keyword, which formats has APPSC really used, and how often?* Generation follows the observed distribution; the fixed rotation is only a fallback where there is no evidence.

THE BUG WORTH REMEMBERING

This engine was **inert for weeks and nothing looked wrong**. `plannedFormatsFor` sat at module scope and called a logging helper defined inside `main()`. So whenever the PYQ layer *did* have evidence, the log line threw a `ReferenceError` into its own `catch`, and the catch returned the rotation. The fallback is a legitimate answer, so every run looked healthy.

Measured on the live database: the keyword `Scheme` has **52 questions of usable evidence** asking for `[direct_recall × 3, negative_statement]`. Every item ever generated had silently received the generic rotation instead.

How many questions an item gets

Not a flat four. The count follows **the number of distinct syllabus units the article feeds**, from a base of 4 up to a cap of 10. An article examinable in six places deserves more questions than one examinable in one, and that is a fact the system already knows by this stage.

Distractor discipline and staleness

- Distractors must be **real-but-wrong** — last year's host, the previous scheme name, the adjacent minister. A distractor that is obviously absurd teaches nothing.
- Every question carries `fact_as_of`, so a stale key is *identifiable* rather than just quietly wrong at revision time.
- An explanation is mandatory.
- Questions are deduped on a normalised hash **against the whole corpus**.
- A unit code returned with a question is canonicalised against `ref_units`, and an unresolvable one is reported rather than stored as a tag nothing can query. This is now the only path by which a model-supplied unit code enters the database.

11. Stages 7–8 — review and publication

Everything above produces *drafts*. The transition to student-visible content is always a human act.

The review queue

Every drafted item lands in Admin → Review queue with its sources, its tags and its correction hits shown **inline**. That inline placement is a deliberate product decision: a review that requires clicking away to check a URL is a review that quietly stops happening. An item with no `prelims_facts` is flagged in red, because nothing can be asked about it.

Publishing a day checks first, then publishes

Publishing a whole day validates every draft in it and reports exactly what blocks it, rather than publishing eight of ten and failing silently on two.

Two gates, not one

WHY QUESTIONS HAVE THEIR OWN STATUS

An item is held by its own `status`. But questions can be *regenerated onto an item that is already live* — so the item's status alone no longer answers "may a student see this". `ca_mcqs.status` is the second gate, and regenerated questions wait under **Admin → Review queue → Questions waiting on review** until approved. Every student-facing read applies both.

Superseding rather than overwriting

When a redraft replaces an item that was already published, the new item records `supersedes` pointing at the old one. The history is kept; the student sees the current position. A published item is never withdrawn by a redraft.

Architecture

The stack, what runs as what, where every file lives, all thirty-five tables grouped by the layer they belong to, and the complete HTTP surface.

12. Stack, processes and the request path

Layer	Choice
Runtime	Node 20
Server	Express 4, <code>better-sqlite3</code> (WAL mode), <code>jsonwebtoken</code> , <code>bcryptjs</code>
Database	One SQLite file — <code>server/data/ca.db</code> . No coupling to any other app's database.
Web	React 18, Vite 5, Tailwind 3, <code>react-markdown</code> with <code>remark-gfm</code> and <code>remark-breaks</code> , <code>react-router-dom</code> 7
PDF handling	Python (PyMuPDF) for layout extraction; Tesseract for OCR where a page has no text layer
Pipeline	Plain Node scripts, run from the command line or spawned by the API

Processes

One process serves both the API and the built frontend. There is no horizontal-scaling story and none is needed: the load is a few hundred students reading a page a day.

The two long pipeline stages — processing an edition and drafting from it — are spawned **out of process, detached**. A `ca_runs` row acts as the drafting lock, and a run still marked running after two hours is closed automatically on the way past.

A RACE WORTH KNOWING ABOUT

The route checks the lock and then spawns the worker; **the worker opens its own run row a few seconds later**. Every click inside that window passes the check, because no row exists yet. Seven phantom runs were created this way in three minutes when a UI fault made the page go blank and the admin clicked repeatedly. They drafted nothing and cost nothing, but they held the per-edition lock until the two-hour sweep.

The request path

```
nginx (TLS)
→ Express
  security headers, hand-written      (CSP with per-script hashes,
                                      HSTS, nosniff, X-Frame-Options DENY,
                                      Referrer-Policy)
  request log                        (no IP, no query string, no email)
  /api/admin/editions → multipart upload, mounted BEFORE express.json
  express.json({ limit: '10mb' })
  /api/health      → item count + uptime, 503 if the DB is unreachable
  /api/auth        → register (throttled), login (throttled), reset, me
  /api             → student content
  /api/topics      → topic graph + syllabus map
  /api/admin       → admin CRUD, queue, publishing
  express.static(web/dist)
  SPA fallback → index.html
  error handler   → 413 oversize upload, 400 unreadable body,
                  500 with no stack in production
```

The edition-upload router is mounted *before* `express.json` deliberately — a 120 MB multipart PDF must not pass through a JSON body parser. nginx needs `client_max_body_size 130M` to match.

13. Repository map

appsc-current-affairs-app/	
├─ README.md	what shipped
├─ SPEC.md	the product vision, and the standing decisions
├─ PLAN.md	the architecture record of the first build
├─ docs/	this document, and the HTML it is built from
├─ ops/RUNBOOK.md	deploy, systemd, nginx, backup, restore, symptoms
├─ scripts/	
│ └─ check-rich-text.js	gate: model-written fields must be rendered
│ └─ check-routes.js	gate: every internal link matches a route
│ └─ check-hooks.js	gate: no React hook after an early return
├─ server/	
│ └─ src/	
│ │ └─ index.js	the one process: headers, routing, static, shutdown
│ │ └─ auth.js	JWT issue/verify, requireAuth, requireAdmin
│ │ └─ db/	
│ │ │ └─ schema.sql	the schema
│ │ │ └─ index.js	connection, WAL, migrations
│ │ └─ lib/	
│ │ │ └─ ingest.js	SECTION 1 — extract, segment, merge, persist
│ │ │ └─ relevance.js	SECTION 2 — the 0-100 score and its breakdown
│ │ │ └─ select.js	STAGE 4 — leverage, rank, adaptive selection
│ │ │ └─ draft.js	SECTION 3 — article → master knowledge object
│ │ │ └─ topics.js	alias matching, shared by topics and units
│ │ │ └─ pyq.js	format mix and recurrence from real papers
│ │ │ └─ quiz.js	paper builder — windows and the 8-format mix
│ │ │ └─ revision.js	Leitner boxes 1-5, 1/3/7/14/30 days
│ │ │ └─ pacing.js	the opt-in reading clock
│ │ │ └─ corrections.js	the known-corrections guard
│ │ │ └─ rateLimit.js	login and signup throttles
│ │ │ └─ passwordReset.js	single-use reset tokens
│ │ │ └─ appTime.js	LOCAL date handling, not UTC
│ └─ routes/	
│ │ └─ content.js	student API
│ │ └─ admin.js	admin API + the validators the pipeline reuses
│ │ └─ editions.js	upload, process, draft, inspect
│ │ └─ topics.js	topic graph + syllabus map
│ │ └─ auth.js	register, login, reset, me
└─ scripts/	
│ └─ seed.js	admin + reference vocabularies
│ └─ reference-data.js	keywords, units, corrections
│ └─ g2-syllabus.js	Group-II unit definitions and aliases
│ └─ g1-prelims-syllabus.js	Group-I Prelims unit definitions and aliases
│ └─ process-edition.js	extract → segment → score (no model)
│ └─ draft-articles.js	select → draft → questions (the paid worker)
│ └─ drop-g1-mains.js	the record of the objective-only migration
│ └─ syllabus-coverage.js	what the map fed, missed, and over-matched
│ └─ requestion-items.js	rewrite an item's questions, priced first
│ └─ preflight.js	what must be true before anyone else can reach it
│ └─ backup.js	online backup, safe while running
│ └─ test-bridge.js	the test suite
└─ web/	
│ └─ src/context/LensContext.jsx	the G1-Prelims/G2/both lens every screen reads
│ └─ src/components/	ItemCard, RichText, McqCard, QuizRunner,
│ │ └─ SyllabusMap, useConfirm, admin/QuestionReview	
│ └─ src/pages/	the student screens
│ └─ src/pages/admin/	the admin screens
└─ public/sw.js	the PWA service worker

content-pipeline/	
np-daily/	the NEWSPAPER lane (primary)
layout.py	PDF → text with bounding boxes, OCR fallback
segment.js	blocks → articles, by the column-ownership rule
genre.js	the paper's own running head → genre
gate-rules.js	the veto lists
profiles.js	per-masthead layout profiles
merge.js	same-event merging
ca-daily/	the WEB lane (predates it, still covers PIB)
lib.js	the ONE model-call implementation
run.js	candidates → triage → draft → MCQs → queue
prompt-draft.txt	the objective-only drafting prompt
prompt-mcq.txt	the 8 formats and the distractor rules
pyq/	past-paper extraction

14. The data model

Thirty-five tables, in six groups. The group tells you the rules that apply — in particular whether a table is **curated** (hand-maintained, precious) or **derived** (rebuildable from scratch, and deliberately disposable).

THE CURATED / DERIVED DISTINCTION

Derived tables — `topic_items` , `topic_units` , `np_article_units` , `np_article_topics` , `pyq_question_topics` — are rebuilt on every re-score. This is deliberate: the matchers *will* improve, and a derived table that cannot be thrown away becomes a liability the moment it disagrees with the code that produced it.

A. Reference vocabularies — seeded, never hardcoded in app logic

Table	Rows	What it holds
<code>ref_keywords</code>	737	Blueprint keyword <i>angles</i> — not facts, but recurring question angles APPSC reuses across years: <code>Appointed</code> , <code>GI tag</code> , <code>Index</code> , <code>Summit</code> . Nine subject lists.
<code>ref_units</code>	52	Every syllabus unit with its code, label, paper, exam, marks and the <i>actual syllabus text</i> . 27 Group-I Prelims, 25 Group-II. Flags: <code>broad</code> and <code>unfeedable</code> .
<code>ref_unit_aliases</code>	934	The words a <i>newspaper</i> uses for a unit, as opposed to the words the syllabus uses for itself. See §17 — this table is why matching works at all.
<code>ref_corrections</code>	4	Facts already found to have gone stale, with the corrected position. Injected into the prompt <i>and</i> checked against every draft.

B. Content — the student-visible material

Table	Rows	What it holds
<code>ca_days</code>	3	One row per calendar day = the daily digest. A day exists even when thin — "the pipeline found little on 14 Aug" is information a student should see, not something to paper over with filler.
<code>ca_items</code>	155	The master knowledge object: the shared body, the static material, and the memorise-this block. 119 published, 36 discarded.

ca_item_keywords	599	Which question angles this item can be tested through.
ca_item_units	409	Every syllabus unit the item feeds, across 46 distinct codes, in both exams.
ca_item_sources	155	Provenance. is_primary flagged so the queue shows which items rest only on secondary reporting.
ca_mcqs	874	The question bank. Carries format , keyword , unit_code , difficulty , fact_as_of and its own status .

C. Student state

Table	What it holds
ca_progress	Marked-read state, and reading_started_at for the pacing clock. Marking read is what unlocks the questions.
ca_attempts	Every answer given, with the session it belonged to.
ca_sessions	Practice papers: scope, label, totals, timing.
ca_revision	Leitner scheduling over both items and MCQs. Polymorphic item_type , so due-queries always INNER JOIN back to the real table and an orphaned row stops appearing rather than erroring.
ca_bookmarks	Saved items.
ca_mcq_flags	Student-reported bad questions. Most of the bank is generated, so the people best placed to catch a wrong key are the students hitting it.
users , password_resets	Accounts and single-use reset tokens. exam_track is g1 / g2 / both .

D. Section 1 — source intelligence

Table	Rows	What it holds
np_editions	4	The newspaper as uploaded: publication, edition, date, stored path, SHA-256, pages, and the run counts that make it auditable — pages_ocr , pages_skipped , articles_found , events , merged . Unique on (publication, date, file hash).
np_articles	411	Every article the segmenter found, kept whether or not it was used. Carries headline, standfirst, byline, dateline, body, page, prominence, extraction mode, genre, the score and its breakdown, and item_id once it becomes a knowledge item.

Why keep the source at all in a topic-centric system? Because a knowledge item must be traceable to the page it came from. Without these two tables the only provenance was a citation string, which cannot answer "show me everything from the 21 August Vijayawada edition", cannot show what a run discarded, and cannot let an admin re-run one page.

E. Section 2 — knowledge intelligence DERIVED

Table	Rows	What it holds
np_article_entities	1,829	People, organisations, places and schemes named. Separate from the article row because one article names many, and "which articles mention APCRDA" is worth being able to ask directly.
np_article_keywords	1,189	Blueprint angles matched <i>before</i> drafting, with <code>in_headline</code> and the <code>pyq_count</code> copied in at scoring time so the score stays reproducible from the row.
np_article_topics	109	Master topics matched through the alias vocabulary.
np_article_units	395	Syllabus units an article touches, decided before any model is asked. A row is written only when the match is already solid.

F. The topic layer, the PYQ layer, and the audit trail

WHY THE TOPIC LAYER EXISTS

Everything in group B is organised by **day**. That is the right shape for reading a day's news and the wrong shape for preparing an exam, because the exam does not ask about a day. It asks about Polavaram.

A topic is the entity a news item is *about*, and it persists. Ten items across eight months mentioning Polavaram are ten updates to one master topic — and until there is a row for the topic itself there is nowhere for the eleventh to attach.

Table	Rows	What it holds
topics	119	Curated master topics with kind, tier and an <code>ap</code> flag.
topic_aliases	441	How the topic is actually written in print. A paper says "APCRDA", "CRDA" and "Capital Region Development Authority" for one body.
topic_items	109	DERIVED Which items touch which topic.
topic_units	182	DERIVED The cross-paper reuse map, and the source of Factor E . A topic inherits the units of every item naming it; <code>weight</code> = how many items support the pairing.
topic_links	20	Curated relations for what item overlap cannot express: Polavaram belongs <i>under</i> AP Bifurcation.
pyq_papers	10	Real past papers: 6 Group-II Screening, 4 Group-II Mains. Carries a <code>notes</code> field, because extraction is imperfect and should say so.
pyq_questions	1,137	Every question extracted, with <code>format</code> in the <i>same vocabulary</i> as <code>ca_mcqs.format</code> . Keeps <code>raw</code> , <code>ocr_confidence</code> and <code>needs_review</code> .
pyq_question_keywords	1,367	Which blueprint angle each question tests.
pyq_question_topics	218	DERIVED Which master topic each question belongs to. The source of <code>topics.tier</code> .

ca_runs	30	One row per pipeline run: window, mode, model, status, candidates, drafted, discarded , approved, log. Also the drafting lock.
---------	----	---

15. The API surface

Authentication — /api/auth

Route	Notes
POST /register	Gated by ALLOW_REGISTRATION , throttled per IP per hour
POST /login	Throttled; keys on client IP, which is why X-Forwarded-For is mandatory behind a proxy
GET /reset/:token · POST /reset/:token	Single-use password reset
GET /me · PUT /me	Profile, exam track, pacing preference, password change

Student content — /api

Route	Notes
GET /meta	Vocabularies the UI needs — buckets, keywords, units
GET /today	Resolves to the <i>latest published</i> digest, not the literal date
GET /days · GET /days/:date · GET /archive	The month-first archive
GET /items/:id	Full item, both lanes; questions included only once unlocked
POST/DELETE /items/:id/read	The unlock gate
POST/DELETE /items/:id/bookmark · GET /bookmarks	Saved items
GET /practice · POST /mcqs/:id/attempt	Paper builder and answering
POST /sessions · GET /sessions	Practice sessions
GET /revision/due · POST /revision/review	Leitner queue
GET /mistakes	Wrong answers grouped by keyword angle
GET /progress · GET /months/:month · GET /search	Progress, monthly revision, search

Topics and syllabus — /api/topics

Route	Notes
GET / · GET /:slug	The topic list and one topic's full dossier
GET /reuse-map	Which topics pay across which objective papers
GET /gaps	Tier-1 topics with no material attached
GET /syllabus · GET /syllabus/:code	The syllabus map grouped by exam, counting items and questions per unit

Admin — /api/admin

Route group	Notes
GET /overview · GET /queue	Dashboard and the review queue
/days — list, create, update, delete, :id/items , :id/publish , :id/unpublish	Publishing a day validates every draft first
/items — create, update, delete, :id/publish , :id/discard , :id/restore , :id/corrections	Discard and restore, never delete by default
/items/:id/mcqs/publish · /items/:id/mcqs/discard · /mcqs/publish-all	The second gate — question review, including Approve all
/mcqs	Manual authoring, first-class
/students · /students/:id/reset-link	Create a student, send them a reset link

Editions — /api/admin/editions

Route	Notes
POST /	Multipart PDF upload; idempotent on file hash
POST /:id/process	Spawns extract → segment → score, detached
POST /:id/draft · GET /:id/draft	Starts a drafting run and reports its progress
GET / · GET /:id · GET /:id/articles/:articleId	Inspect an edition down to a single article
DELETE /:id	Remove an edition

The intelligence layers

The ten-layer model the product spec sets out, what is genuinely built against each, and then the three pieces of machinery that do the real work: the syllabus map, the PYQ engine, and the topic graph.

16. The ten layers

#	Layer	Status	Where it lives / what remains
1	Newspaper ingestion	BUILT	np_editions , np_articles , lib/ingest.js , routes/editions.js , np-daily/
2	Duplicate intelligence	PARTIAL	Same-edition merge works. Cross-language merge is out of scope by decision (English only)
3	APPSC relevance filter	BUILT	lib/relevance.js — 0–100 on five factors, CRITICAL–LOW bands, vetoes first
4	Blueprint keyword engine	BUILT	737 ref_keywords , np_article_keywords , np_article_entities
5	PYQ intelligence	BUILT	pyq_questions (1,137 across 10 papers), plannedFormats() , and topic recurrence
6	Group-II output engine	BUILT	Notes → keyword → PYQ pattern → MCQs → revision
7	Group-I output engine	BUILT	Prelims only. 27 units, its own lane in the syllabus map and the lens
8	AP knowledge graph	BUILT	topics / topic_aliases / topic_items / topic_links , with a UI at /topics
9	Unified note engine	BUILT	One master note → two syllabus views. The article→note bridge, lib/draft.js
10	Paper-I essay engine	REMOVED	Served Group-I Mains only. Deleted 23 Aug 2026 — see §33

17. The syllabus map and its aliases

THE INSIGHT THE WHOLE MAP RESTS ON

The syllabus says *"Distribution of Legislative and Executive Powers between the Union and the States"*. The newspaper says *"Centre-State"*, *"concurrent list"*, *"Article 246"*. **Matching the syllabus's own phrasing would match almost nothing.**

So `ref_unit_aliases` holds 934 newspaper-side phrasings across 52 units — and it is the single highest-leverage file in the repository. Improving it costs nothing and changes what every future article scores.

Unit flags that keep the map honest

Flag	Effect
exam	g1p or g2 . This is what the lens switches on, and what Factor E counts distinct papers within.
broad	A unit so wide it matches nearly everything — e.g. the 30-mark general current-affairs paper. Excluded from leverage, scoring evidence and coverage reports.
unfeedable	A unit no newspaper article can ever feed — mental ability, the language papers. Excluded the same way.

The standing coverage report

`syllabus-coverage.js` is run after each edition. It answers three questions the map cannot answer about itself: what the map fed, what was never fed, and what over-matched. Currently **110 articles score 45 or more; 93 map to a syllabus unit and 17 map to none**, and only three units have never been fed — G1P-A2 , G2-P1-U1 , G2-P1-U4 .

THE LIVE CONCENTRATION PROBLEM, AND WHY IT IS PRICED RATHER THAN FIXED

G2-P1-U7 is fed by **61 articles against a median of 7**, and 34 of those rest on a single matched term — mostly "Supreme Court" or "High Court" appearing in a story that is really about something else. A unit matching a third of the paper cannot answer "which items feed it", which is the only question it exists to answer.

The report prices the obvious fix rather than applying it, because dropping the weak term strands articles and written questions. Its own advice, and the right reading: a high stranding number usually means the *other* units are too narrow rather than this one being too broad. Widen those first; it costs nothing already written. **This is a judgement call for the product owner, not a bug to be silently patched.**

18. The PYQ engine

What the commission has actually asked, held as data: **1,137 objective questions across 10 real papers**, each with its format classified into the same eight-value vocabulary the generator uses. That deliberate sameness is the entire point — it lets the system compare *what was asked* with *what is being generated*, per keyword.

Given the blueprint keyword "Scheme":
52 past questions of evidence
observed mix → [direct_recall, direct_recall, direct_recall, negative_statement]

Generation follows that, not a rotation.

Corpus	Papers	Questions
Group-II Screening	6	537
Group-II Mains	4	600
Total	10	1,137

WHAT THE GROUP-I MAINS CORPUS WAS DOING, AND WHY LOSING IT COSTS LITTLE

20 descriptive papers and 252 question slots were seeded here and are now removed. A descriptive paper cannot tell you which FORMAT a topic is asked in, so that half only ever contributed *recurrence* — and it contributed it to an app that no longer sits the paper. `topics.tier` is now derived from recurrence in the 1,137 real objective questions instead, which changed the tier of 23 topics.

One seeding decision from that era is worth keeping in mind if a descriptive corpus is ever re-added: **one row per question NUMBER, not per alternative.** 1 (a) ... OR (b) ... is one slot a candidate answers once, and counting alternatives separately would double every topic count and inflate recurrence.

19. The topic knowledge graph

119 curated master topics, 441 aliases, 20 curated relations, and a derived reuse map of 182 topic↔unit pairings across 109 item matches.

What it buys

- **Accumulation.** The eleventh Polavaram story attaches to the previous ten instead of starting a new page.
- **Cross-paper reuse as a query.** "Which papers does this topic serve?" is answered from `topic_units`, with `weight` separating a real reuse from one item's stray tag. **This is now the source of Factor E.**
- **Tier from measured recurrence** rather than hand assignment.

Two precision rules that were needed

The matcher originally returned 73 matches where 59 were correct. Two rules fixed it, and both are the same idea as the syllabus-unit evidence rule in §7: **a match must be earned** — named in the headline, or supported by more than one distinct alias, or carried by a phrase specific enough to stand alone. A single common word is not evidence of what a story is about.

A NUMBER THAT MOVED FOR A GOOD REASON

Topic matches fell from 152 to 109 when the Mains layer went. That is not a regression: the eight-section Mains note was most of the text being matched against, and a topic is now matched on what the objective lanes actually publish.

The product

What a student and an admin actually see, and the complete list of gates standing between a model's output and a student's screen.

20. Student screens

Screen	What it does, and the decision behind it
Today /	Resolves to the <i>latest published</i> digest, not the literal date — a day the pipeline hasn't run should not look like a broken app. Items grouped by bucket, importance-ranked, with the lens toggle in the header switching which syllabus each card is filed against.
Day /day/:date	One digest in full.
Item /item/:id	Notes → static linkage → static notes → the two syllabus lanes → sources → questions. Questions unlock on <i>Mark as read</i> .
Archive /archive	Month-first, because current affairs is revised by month, not by day. Density-shaded grid; filter by bucket, keyword, unit, importance.
Month revision /month/:month	Auto-compiled month compendium, a monthly test, and "what changed since you last read this".
Practice /practice	Scoped by window ("last 7 days", "this month") rather than by topic — news is revised by date. Mixes the paper deliberately across formats instead of serving the bank in whatever order SQL returns it. Reports how many questions are still locked, so a thin paper reads as "go and read" rather than "the bank is empty".
Revision /revision	Leitner due-queue over items and questions together.
Mistakes /mistakes	Grouped by keyword angle, because "you keep missing Appointed questions" is actionable and a flat list of wrong answers is not.
Topics /topics , /topics/:slug	The knowledge graph, the reuse map, the gaps, and the syllabus map by exam and unit.
Saved · Progress · Search · Profile	Bookmarks; streak, accuracy trend, coverage heatmap, AP share; full-text search; account settings including exam track and the pacing switch.

The lens

`LensContext` holds `g1` / `g2` / `both`, and every screen reads it. It defaults from the user's `exam_track`. The stored key stays `g1` rather than becoming `g1p` deliberately: it is written into every user's `localStorage` and into `users.exam_track`, and renaming it would silently reset the lens for everyone who has already chosen one — for a label change.

The MCQ unlock

WHY QUESTIONS ARE HIDDEN UNTIL THE ITEM IS MARKED READ

A question answered before the notes teaches the answer, not the topic.

But "mark as read" is a button, and a button takes half a second — the gate records an intention, not a reading.

Pacing makes the gate mean what it says: the clock starts server-side when the item is opened, and the questions open when the clock has run.

It is **opt-in and stays opt-in**, because a discipline imposed on somebody is just an obstacle. A candidate revising the night before wants to move at their own speed through material they have already read; a candidate meeting an item for the first time does not. The same person is both, on different days.

Spaced revision

Five Leitner boxes at 1 / 3 / 7 / 14 / 30 days. A correct answer promotes; a miss drops straight back to box 1. New cards start at box 1 due *tomorrow*, so they appear as "due soon" rather than retroactively overdue. Dates are **local** YYYY-MM-DD strings, not UTC — as UTC, the day boundary fell at 05:30 in the morning for every Indian student.

21. Admin screens

Screen	What it does
Dashboard /admin	Counts across the queue, discards (shown as a positive signal), runs, and content health.
Editions /admin/editions	The primary lane. Upload the day's PDF, watch processing, then drafting — both spawned out of process. The panel counts candidates with the <i>same selector the drafter uses</i> , so the number shown is the number you get.
Review queue /admin/queue	The safety gate. Sources, tags and correction hits inline. Two tabs: items awaiting review, and questions waiting on review — the second gate, with Approve all.
Days /admin/days	Create, edit, publish and unpublish digests. Publishing validates the whole day first.
Item editor /admin/days/:dayId	Full manual authoring — the same form the pipeline fills. Syllabus units are picked here; question editor with format picker and live preview.
Students /admin/students	Create a student and send a reset link — the alternative to open registration.

TWO UI FAILURES WORTH GENERALISING

"Approve all" did nothing when clicked, with no error. The app used `window.confirm`, and embedded browsers suppress it and return `false` — so the code correctly did nothing, forever. All eight native dialogs were replaced with an in-page component. A browser primitive that can be silently disabled is not a safe place to put a decision.

Every edition-detail page rendered blank. One hook was called after an early return, so the loading render called four hooks and the loaded render called five. React counts them, throws error #310, and unmounts the entire tree — navbar and all. Nothing else complains: every route returns 200, the API is fine, the server log is clean. It is now caught by a static gate.

22. The gates

Everything that must pass before a student sees a word. Layered deliberately, because the single most expensive failure mode for this app is a confidently wrong current-affairs fact — which, unlike a static-subject error, cannot be caught against a textbook.

#	Gate	What it stops
1	The veto list	Crime, sport, letters, archive reprints — excluded before scoring, so they cannot be outvoted by it.
2	The selector	Anything with no syllabus connection, and anything the scorer called junk regardless of leverage.
3	The facts rule	An item with no memorise-this block — nothing can be asked about it. Applied in the drafter before anything more is spent, and flagged in the review queue.
4	The corrections guard	A superseded position being re-filed. Runs at three points — pipeline write, queue render, editor request. It <i>annotates rather than blocks</i> , because only a person can tell "an item about the 16th Finance Commission" from "an item that assumes the 15th is still operative".
5	<code>validateItem</code>	Publishing without notes; a missing prelims-facts block; an unknown bucket or importance.
6	<code>validateMcq</code>	A question under 10 characters; a missing option; duplicate options; an invalid key; an unknown format; a missing explanation .
7	Corpus-wide dedupe	A question already asked, matched on a normalised hash across the whole bank rather than the current run.
8	Item status	A person publishes. Nothing publishes itself.
9	Question status	Questions regenerated onto an already-published item wait for their own approval.
10	The read/pacing gate	A question shown before the note it tests.

Operations

How to set it up, run a day, configure it, deploy it, back it up, and what the automated checks cover.

23. Setup, running, and the daily commands

FIRST-TIME SETUP

```
cd server && npm install && npm run seed
cd ../web && npm install && npm run build
node server/scripts/seed-g2-syllabus.js
```

RUNNING

```
cd server && npm start      # one process, API + built frontend, port 4100
cd web && npm run dev        # frontend only, proxies /api to 4100
```

THE DAILY NEWSPAPER LANE

```
# 1. Upload the PDF in Admin → Editions (or run the two steps directly)

node server/scripts/process-edition.js <editionId>      # extract → segment → score
node server/scripts/draft-articles.js <editionId> --plan # see the selection, free
node server/scripts/draft-articles.js <editionId>      # notes + questions

# 2. Review and publish in Admin → Review queue

# 3. Standing reports
node server/scripts/syllabus-coverage.js              # fed, never fed, over-matched
node server/scripts/requestion-items.js --dry-run     # re-tag questions, priced first
```


USEFUL FLAGS ON THE DRAFTING WORKER

Flag	Effect
<code>--plan</code>	Print the selection and stop. No model calls, no cost, ~0.4s.
<code>--dry-run</code>	Draft everything and print it, but write nothing. Costs full price.
<code>--max N / --min N</code>	Cap and floor on the adaptive selection. <code>--max</code> wins over the floor.
<code>--min-score N</code>	Restore the old flat threshold for one run.
<code>--article ID,ID</code>	Hand-pick, whatever they score. The gate is the person.
<code>--redraft</code>	Include articles that already produced an item.
<code>--mcqs-per N / --no-mcqs</code>	Base question count, or notes only.
<code>--limit N / --model <id></code>	Stop after N; override the model.

THE WEB LANE (PIB), WHICH PREDATES THE NEWSPAPER LANE

```
node content-pipeline/ca-daily/daily.js --date 2026-08-21 --dry-run
node content-pipeline/ca-daily/daily.js --date 2026-08-21
```

Four stages: **discover** → **shortlist** → **fetch** → **draft**. Two decisions carry most of its value. **Discovery uses no model** — asking a model what happened invites invented URLs; asking PIB for its own index returns the actual list. And **the shortlist runs on headlines, not bodies** — fetching 40 releases to throw 32 away is waste.

24. Configuration

Variable	Default	What it does
<code>JWT_SECRET</code>	—	Set it. Without it the server falls back to a secret published in the repository, and anyone who has read the source can mint an admin token. 48 random bytes. Changing it later logs everyone out.
<code>NODE_ENV</code>	unset	Set to <code>production</code>. Not cosmetic — it is what turns the missing-secret <i>warning</i> into a <i>refusal to start</i> , and what stops Express sending stack traces.
<code>ALLOW_REGISTRATION</code>	1	Whether anyone with the URL can sign themselves up. Every item cost money to draft, and the admin screen already has the other model — create the student, send a reset link.
<code>MAX_SIGNUPS_PER_HOUR</code>	5	Per IP.
<code>PORT</code>	4100	
<code>CORS_ORIGINS</code>	empty	Leave blank. The frontend is served by the same process and needs no CORS at all.

OPENAI_API_KEY	—	Required only for drafting. The app itself runs without it.
OPENAI_MODEL	gpt-5.6-luna	The drafting model.
OPENAI_TIMEOUT_MS	180000	How long one call may take before it is abandoned and retried.
ALT_BASE_URL / ALT_API_KEY / ALT_MODELS	unset	A second provider for the cheap job only. See the warning below.
LOG_ALL_REQUESTS / SLOW_REQUEST_MS	0 / 1000	The log records no IP, no query string and no email by design.
NP_PYTHON	python	Interpreter used for layout extraction.

WHICH JOBS ARE SAFE TO MOVE TO A CHEAP MODEL

Shortlisting is. It is judgement over forty supplied headlines, no recall involved, and a wrong call loses one article the next day's paper will cover anyway.

Drafting is not. It writes the static notes — Articles, sections, landmark cases, AP schemes — from the model's own knowledge, and a cheap model that invents there teaches a candidate something false with total confidence. Measured August 2026: one popular budget model hallucinated on 96% of knowledge questions. Keep drafting on the better model; the whole month costs about \$4 either way.

25. Preflight, backup, deploy, restore

Preflight — what must be true before anyone else can reach it

```
npm --prefix server run preflight
```

It checks the things that leave the app *working perfectly while being totally wrong*: an unset `JWT_SECRET`, a `web/dist` older than `web/src`, an unopenable database, an admin still on the seed password, published items with no visible questions. It exits non-zero, so it can gate a deploy.

CURRENT PREFLIGHT RESULT — 23 AUGUST 2026

8 passed, 2 warnings, 0 failures.

PASS · `.env` present · `JWT_SECRET` set, 64 characters · `CORS_ORIGINS` empty · `web` build newer than `src` · database opens, integrity ok · 2 admins, none on a seed password · every published item has questions a student can see · backups present.

WARN · `NODE_ENV` is unset — set it to `production` .

WARN · registration is OPEN — anyone with the URL can create an account.

Both warnings are decisions waiting on the product owner, not defects.

Backup

```
npm --prefix server run backup      # --verify opens the copy afterwards
```

Backups use SQLite's **online backup API, not a file copy**. A `cp` of a WAL-mode database can silently produce one missing its most recent writes — and it opens cleanly, so you find out during a restore.

Deploy

One box, one process, one SQLite file. `systemd` with `Restart=always`, `ProtectSystem=strict` and `ReadWritePaths` limited to the data directory; `nginx` in front terminating TLS with `client_max_body_size 130M`. `X-Forwarded-For` is **required, not optional**: the app sets `trust proxy` and the login throttle keys on client IP — without it every request looks like `127.0.0.1` and the throttle becomes one shared bucket for the whole internet.

A WINDOWS CAVEAT THAT MATTERS FOR THIS MACHINE

On `SIGTERM` the process stops accepting connections, lets in-flight requests finish, checkpoints the WAL and closes the database. **On Windows that path does not run** — `Stop-Process` and Task Manager both terminate outright rather than signalling.

It is not fatal: SQLite recovers the WAL when the next process opens the file. But it is exactly the window in which a `cp`-style backup produces a copy missing its most recent writes. On Windows: stop it with `Ctrl+C` in its own console, and take backups *only* through `server/scripts/backup.js`.

26. Tests and static gates

```
npm --prefix server test
```

The bridge suite plus three static gates. **146 of 146 tests pass as of 23 August 2026**. Every gate was written after the fault it catches had already shipped:

Gate	Catches
<code>check-rich-text.js</code>	A model-written field printed raw, so its <code>**bold**</code> shows as literal asterisks — and a self-closing <code><RichText /></code> , which takes <code>children</code> and therefore renders nothing . That one blanked an entire review screen.
<code>check-routes.js</code>	An internal link pointing at no declared route — currently checking against 26 declared routes. The router answers with the 404 page rather than an error, so nothing else notices.
<code>check-hooks.js</code>	A React hook called after an early return, which throws error #310 and unmounts the whole tree. The user sees a white page and a minified console error naming no component.

The suite's coverage is heaviest where the reasoning is subtlest. Sixteen tests cover the adaptive selector alone, including *"a high score with no syllabus unit is NOT drafted"* and *"leverage cannot rescue an article the scorer called junk"*. Two now pin the objective-only prompt: *"no descriptive unit survives anywhere in the vocabulary"* and *"the prompt tells the model the units are settled elsewhere"* — a regression there would be expensive rather than merely wrong, because the unit listing was the largest block in a paid call.

Constraints, limits and state

The reference card. Standing decisions, enforced invariants, honest limitations, the open backlog, the live numbers, and what the objective-only migration took out.

27. Standing decisions

These are settled. Reopening one is a product decision, not a code change.

Decision	Value	Since
Repository	Extend <code>appsc-current-affairs-app</code> . Do not start a new one.	21 Aug 2026
<code>appsc-group2-prep-app</code>	Never modified. Sibling, standalone, separate database.	21 Aug 2026
Input	Admin uploads a PDF; everything flows from it.	21 Aug 2026
Newspaper sources	English only — The Hindu. Eenadu / Telugu ingestion is out of scope.	22 Aug 2026
Exams served	OBJECTIVE ONLY — Group-II Screening, Group-II Mains, Group-I Prelims. Group-I Mains is out of scope and its layer has been removed.	23 Aug 2026
Publishing	Always manual. The drafting is worth automating; the approving is not.	21 Aug 2026
Login	No shared login with the sibling app. Standalone means standalone.	21 Aug 2026
Spec authority	<code>SPEC.md</code> is the authoritative product record. Anything agreed in conversation is written there <i>before</i> it is built.	22 Aug 2026

TWO DISTINCTIONS THAT MUST NOT BE COLLAPSED

English-only removes Eenadu as an input newspaper. It does not remove Telugu as exam content. AP History, Society and Culture are full of Telugu topics, and `relevance.js` must keep matching "Telugu literature", Telugu dynasties and the like.

Objective-only removes Group-I MAINS. It does not remove Group-I. Group-I Prelims is served, and its 27 units are more than half the syllabus map.

28. Invariants the code enforces

1. **No facts, no item.** An item nothing can be asked about is turned away in the drafter and flagged in the queue.

2. **Discard is a row, never a deletion.** Articles and items alike. A segmenter that quietly loses a column looks identical to one that works, and the only defence is being able to read what it decided.
3. **Every question carries an explanation and a `fact_as_of` date.**
4. **Nothing publishes itself.** Two status gates, both human.
5. **Print items carry no invented URL.** Citation from the edition row; `is_primary = 0` ; `needs_verify = 1` .
6. **Derived tables are rebuildable.** Any table the matchers produce can be dropped and regenerated.
7. **Scores store their breakdown.** A number nobody can decompose is a number nobody trusts.
8. **Shared libraries, not parallel implementations.** One ingest, one drafting insert, one model-call implementation, one alias matcher.
9. **The model is never asked what code already settled** — not the bucket, not the keywords, not the topics, and now not the syllabus units.
10. **An article with no syllabus connection is never drafted automatically.**
11. **Dates are local, not UTC.**
12. **Uploads are idempotent on file hash.**

29. Known limitations

Limitation	Detail
One publication, one language	The segmenter is tuned to The Hindu's layout: it reads the paper's own running head to tell an opinion page from a news page. Another masthead needs a profile in <code>np-daily/profiles.js</code> .
<code>G2-P1-U7</code> over-matches	Fed by 61 articles against a median of 7, 34 of them on a single term. The fix is priced in every coverage run; the decision is the owner's.
The app has no crawler	Nothing in the codebase hits the web on a schedule. A scraper cannot tell a superseded figure from a current one, and this is the material where that mistake is unrecoverable.
Automated discovery covers PIB only	AP government orders, PRS Bill summaries, RBI documents and newspaper reporting still need the manual route. RBI's document server is CAPTCHA-gated to automated clients.
The drafting lock has a race	The route checks it; the worker registers it seconds later. Rapid repeat clicks create phantom runs that hold the lock until the two-hour sweep. They cost nothing but block drafting meanwhile.
No shared login	With <code>appsc-group2-prep-app</code> . By decision.
Service worker in the in-app browser	It serves correctly and works in a normal browser; the embedded pane's sandbox refuses the registration.
The value compounds with daily runs	Four editions in, Practice has 874 questions and the map covers 46 units. A month in, per-unit coverage becomes worth <i>reading</i> rather than worth <i>checking</i> .

30. Open backlog

Item	Detail
The upload form does not read the date from the filename	A file named <code>...Full Edition 10~08~2026.pdf</code> uploaded on the 23rd is filed as the 23rd unless the admin changes the date by hand. It happened once and was caught by reading the paper's own weekday. Defaulting the field from the filename would close it — see §31.
Syllabus vocabulary gaps	17 articles score 45+ and match no unit. Each is either a missing alias or a genuine non-match, and telling them apart is a short job with a large payoff.
Three units never fed	<code>G1P-A2</code> , <code>G2-P1-U1</code> , <code>G2-P1-U4</code> — all AP/ancient history, genuinely rare in a daily paper.
Duplicate topics	From merging the news vocabulary with the blueprint vocabulary. Needs a human merge.
PYQ extraction is incomplete	Several past-paper PDFs still have only partial extraction. The filter is fixed; a full run has not been done.
119 published items were drafted under the old prompt	Deliberately left alone. Their Mains fields were stripped; their notes, facts and questions are untouched. New editions get the leaner objective-only drafting.
<code>seed-content.js</code> carries dead keys	The seeded digests still set <code>g1_*</code> fields. They are ignored on insert, so it works — it is untidy rather than broken.
<code>np-daily/README.md</code> is stale	It still says <code>np_editions</code> does not exist.

31. Current content state

Read directly from `server/data/ca.db` , 23 August 2026.

4 EDITIONS	88 PAGES	411 ARTICLES	119 PUBLISHED ITEMS
874 QUESTIONS	690 CARRY A UNIT	52 UNITS MAPPED	35 TABLES

Source

Edition	Pages	Articles	Note
The Hindu, Vijayawada — 10 Aug 2026	24	97	Catch-up edition, not yet drafted
The Hindu, Vijayawada — 21 Aug 2026	28	121	
The Hindu, Vijayawada — 22 Aug 2026	18	95	
The Hindu, Vijayawada — 23 Aug 2026	18	98	

A DATING ERROR WORTH RECORDING, BECAUSE IT WAS NEARLY INVISIBLE

The 10 August edition was uploaded on 23 August with the form's date left at today, so it sat in the database as a second 23 August edition. Nothing about it looked wrong: it processed cleanly, scored normally, and appeared in the list beside the real one.

It matters because **print citations are built from the edition row**, so every item drafted from it would have cited "The Hindu (Vijayawada), 2026-08-23" — a date the story was not published on, on an app whose whole discipline is that a fact carries the date it was true. The date also decides which digest the items land in.

It was caught by reading the filename against the content — the paper reports events "on Sunday", and 10 August 2026 is a Monday — and corrected before anything was drafted, so nothing had to be re-paid for. **The upload form should default its date from the filename where the filename carries one**; that is the fix that stops it recurring, and it is not yet built.

What happened to the 411 articles

Status	Articles
not selected (new)	171
drafted into an item	119
discarded with a reason	64
relevant, not yet drafted	43
merged as duplicates	14

Published items by bucket

Bucket	Items	Share
national	67	56%
andhra pradesh	45	38%
international	4	3%
dynamic	3	3%

Reference data seeded

Vocabulary	Rows	Notes
Keyword angles	737	Nine subject blueprints
Syllabus units	52	27 Group-I Prelims · 25 Group-II
Unit aliases	934	Newspaper-side phrasings
Master topics	119	with 441 aliases and 20 curated links
Past-paper questions	1,137	across 10 objective papers
Known corrections	4	Labour Codes · Amaravati · 16th FC · Census 2027

Verification status

Check	Result
Test suite	146 / 146 passing
Static gate — rich text	PASS
Static gate — routes	PASS — 26 declared routes
Static gate — hooks	PASS — no hook after an early return
Preflight	8 passed, 2 warnings, 0 failures
API sweep	23 endpoints, all 200
Factor E coverage	75 articles earn it, against 26 under the old evidence

32. Decision log — failures that shaped the design

Almost every rule in this document exists because something went wrong first. Kept here because the reasoning is more portable than the fix.

What happened	What it taught, and what changed
A verification pass found 4 of 9 blueprint facts stale in 15 months, 3 of them Tier-1	Staleness is the app's defining risk, not a corner case. Produced <code>ref_corrections</code> , the three-point guard, and <code>fact_as_of</code> on every question.
29 articles lost in one drafting run to <code>fetch failed</code>	The retry gave up on attempt 1 because the error carried no <code>retryable</code> flag. Now a classifier walks the whole cause chain, with backoff, jitter, a timeout, and a second-pass queue.
A killed run lost 13 paid-for drafts	They were held in memory until the end. Now written per record.
The PYQ format engine was inert and every run looked healthy	A <code>ReferenceError</code> in a log line fell into its own <code>catch</code> , which returned the legitimate fallback. Dependencies are now parameters.
313 of 314 datelines wiped by running a backfill twice	The second pass derived from already-stripped bylines. A derivation that consumes its own output is not idempotent.
A whole review screen rendered blank	<code><RichText text={...} /></code> — the component takes <code>children</code> . Became a static gate.
Every admin edition page rendered blank	One hook called after an early return; React counted more hooks than the previous render and unmounted the tree. Became a second static gate.
"Approve all" did nothing, silently	Embedded browsers suppress <code>window.confirm</code> and return <code>false</code> . All eight native dialogs replaced.
Seven phantom drafting runs in three minutes	The blank page above made the admin click repeatedly, and the lock is registered by the worker rather than the route. They drafted nothing, but held the lock.
A blanket <code>DELETE FROM ca_progress</code> removed real reading history	Restored from backup. Test-data cleanup is now scoped to the probe account explicitly.

The syllabus addendum contradicted the base prompt	Two instruction blocks in one prompt must be checked against each other.
The unit vocabulary contradicted the instructions	The model returned 14 units, then 3 objective and 0 descriptive. Narrowing the vocabulary fixed it; removing it entirely made the contradiction impossible.
--max 10 returned 12	The default minimum silently overrode the explicit cap. A flag that is quietly ignored is worse than one that errors.

THE METHOD THESE HAVE IN COMMON

Measure, don't assert. Every threshold in this system — 45, 32, 12, 35, the 55/45 weighting, the four-unit flattening point, the Factor E rescale — was set by running the rule over real articles and reading what it did, then adjusted when the output disagreed with judgement. None is a guess that survived because nobody checked.

33. What the Group-I Mains removal took out

Performed 23 August 2026 by `server/scripts/drop-g1-mains.js`, which is kept in the repository as the record. It is re-runnable and prints a before/after.

Removed	Scale	What it was for
11 g1_* columns on ca_items	154 items	The eight-section Mains note: theme, why-in-news, background, AP angle, linked instruments, essay bridges, way forward, plus THE FACT and THE ANGLE
ca_essay_questions	461 rows	Paper-I essay prompts, direct and indirect
ca_item_dimensions	605 rows	The seven-dimension coverage tags
ca_item_themes	425 rows	Bank-review themes
topic_evidence	149 rows	Mains recurrence — and the sole source of Factor E
ca_user_cards , ca_skeletons	0 rows	The personal Q/D/E/S bank and answer skeletons — never used once
Descriptive ref_units	54 units	The five written papers. They had 0 aliases — never matched from newspaper text, only offered to the model as a vocabulary
ca_item_units rows on those units	641 rows	Mains routing
Group-I Mains PYQ corpus	252 q / 20 papers	Descriptive past papers
The two angle triggers	2	Blocked publishing to the Mains lane without an argument
bankReview.js , Banks.jsx , G1Note.jsx , 4 seeders	7 files	The bank review, its screen, the note renderer, and the Mains seed scripts

What was rebuilt rather than deleted

Piece	Old source	New source
Factor E, 15 of 100	topic_evidence — a blueprint	topic_units ⇌ ref_units , counting distinct objective papers
topics.tier	Hand-assigned, informed by the blueprint	Recurrence in 1,137 real Group-II questions. 23 topics changed tier
The discard rule	No angle, no Group-I lane	No prelims facts, no item
The lens	Written paper vs ticked paper	Two published syllabi, both ticked

34. Glossary

Term	Meaning
Blueprint keyword	A recurring <i>question angle</i> APPSC reuses across years — Appointed , GI tag , Index . Not a fact.
Bucket	One of four: international, national, ap, dynamic. <i>Dynamic</i> is the fast-changing edge of another subject — a fresh GI tag, a rate decision.
Composite score	The 0–100 relevance number from the five weighted factors.
Dateline	Where a story was filed from. The strongest single signal that a story is an AP story, even when its text never names the State.
Derived table	A table the matchers produce and that can be dropped and rebuilt at any time. The opposite of curated.
Digest / day	One ca_days row — a calendar day's published items.
Edition	One uploaded newspaper: publication + edition + date + file hash.
Genre	News, editorial, opinion — read from the paper's own running head, not classified.
Item	The master knowledge object. One ca_items row, routed to both syllabi.
Lane	Group-I Prelims or Group-II — the two syllabi one item is routed into.
Lens	The student's G1-Prelims / G2 / both toggle. Changes which syllabus units each card shows.
Leverage	0–100, from the count of distinct syllabus units an article feeds, plus a bonus if a unit is named in the headline.
Prelims facts	The memorise-this block. The examinable residue of an item, and — since every paper is objective — the whole output.
Prominence	Headline size as a multiple of body size — the editor's own judgement of importance, free and requiring no model.
PYQ	Previous Year Question. Here, 1,137 of them held as structured data.

Rank	$0.55 \times \text{leverage} + 0.45 \times \text{composite}$. What actually orders the selection.
Static linkage / static notes	The field naming the standing syllabus topic a news item updates, and the standing material itself — what stops current affairs being a separate subject.
Tier	A topic's priority, 1–3, derived from measured recurrence in the objective papers.
Unit / unit code	A syllabus unit, e.g. G2-P1-U7 or G1P-B2 . The atom of the syllabus map.
Unfeedable / broad	Unit flags. <i>Unfeedable</i> : no article can ever feed it (mental ability, languages). <i>Broad</i> : it matches nearly everything and so proves nothing.

Compiled 23 August 2026 from the source tree and the live database of `appsc-current-affairs-app` . Every figure in Part VII was read from the running system rather than from documentation. This version supersedes Version 1.0, which described the system before the Group-I Mains layer was removed; where the two disagree, this one is current.